Tablut Player VikingAB — Documento tecnico per la presentazione

Nov 11, 2026

1. Panoramica

Il nostro giocatore è un agente di **ricerca avversaria**: minimax con potatura alpha-beta, approfondimento iterativo e una funzione di valutazione lineare sulle caratteristiche della posizione. È scritto in Java, parla con il server arbitro ufficiale in JSON su TCP e si chiama `VikingAB` (il server ne tiene le prime 10 lettere/cifre).

Cosa chiede la consegna

- Giocare a Tablut con le regole Ashton (scacchiera 9x9) contro il server Java del repo AGalassi/TablutCompetition.
- Al massimo 60 secondi per mossa: timeout o mossa illegale = sconfitta.
- Nessun calcolo durante il turno dell'avversario.
- Risorse previste: 4 CPU, 8 GB di RAM, nessuna GPU, nessuna rete a runtime (i dati hardware possono cambiare).
- Avvio tramite `runmyplayer.sh RUOLO TIMEOUT IP`, tutto in `/home/tablut/tablut`.
- Punteggio del torneo: 3 punti per vittoria, 1 per pareggio, andata e ritorno (una partita da Bianco e una da Nero).

Perché la ricerca avversaria

Tablut è un gioco a due giocatori, deterministico, a informazione perfetta e a somma zero: è il caso d'uso classico di minimax con alpha-beta visto a lezione. Il fattore di ramificazione è contenuto (decine di mosse legali per posizione), quindi con 60 secondi e una buona potatura si arriva a profondità utili. La pianificazione (STRIPS, POP, Graphplan) non è adatta, perché c'è un avversario che reagisce a ogni nostra mossa.

Stato attuale (v1)

- Il motore delle regole è stato confrontato con quello del server su circa 20 milioni di coppie (da, a): 0 differenze.
- Nelle partite di auto-gioco con il server vero come arbitro: 0 mosse illegali, circa 1,85 milioni di nodi al secondo e profondità media circa 7,8 con 300 ms per mossa (misure nel nostro ambiente di sviluppo, non nella VM ufficiale).
- La **forza di gioco non è ancora calibrata**: i pesi della valutazione sono scelti a mano, e in alcune prove di auto-gioco il Nero vince più spesso del Bianco. Il torneo conta entrambi i colori, quindi è il punto da migliorare.

2. Le regole che vale davvero il server

Il nostro motore replica esattamente le regole del server (`GameAshtonTablut.java`), comprese alcune particolarità che differiscono dalle regole "da manuale" e dalle slide: l'arbitro è il codice del server, quindi è quello a cui ci siamo adeguati.

Aspetto	Cosa fa il server
Coordinate	La lettera è la colonna (a–i), il numero è la riga (1–9). Il trono è e5.
Trono vuoto	È un ostacolo per tutti, re compreso: non ci si ferma e non lo si attraversa. Il re quindi non può tornarci.
Accampamenti	16 caselle (a4 a5 a6 b5, d1 e1 f1 e2, i4 i5 i6 h5, d9 e9 f9 e8). Vi entra o li attraversa solo un pezzo che inizia la mossa già dentro un accampamento; da accampamento a accampamento al massimo 5 caselle.
Vittoria del Bianco	Il re raggiunge una qualunque casella del bordo. Gli accampamenti sul bordo non sono raggiungibili, quindi in pratica sono 20 caselle.
Cattura di un soldato nero	Un soldato bianco o il re, muovendo accanto a un nero, lo cattura se dall'altra parte c'è un bianco, il re, il trono vuoto o un accampamento. Fanno eccezione i 4 accampamenti centrali di bordo (a5, i5, e1, e9). Il re cattura quando muove.
Cattura di un soldato bianco	Un nero che muove accanto a un bianco lo cattura se dall'altra parte c'è un nero, il trono (anche occupato) o uno qualunque dei 16 accampamenti.
Cattura del re	Sul trono: 4 neri attorno. Accanto al trono: neri sui 3 lati non-trono. Altrove: 2 lati opposti, nero o accampamento.
Pareggio	Uno stato (scacchiera + turno) già visto dall'ultima cattura è pareggio. La lista si azzera a ogni cattura; lo stato iniziale non è nella lista.
Errori	Zero consentiti per impostazione predefinita: una mossa illegale fa perdere la partita.
Tempo	Il timeout del server ha granularità di 1 secondo.

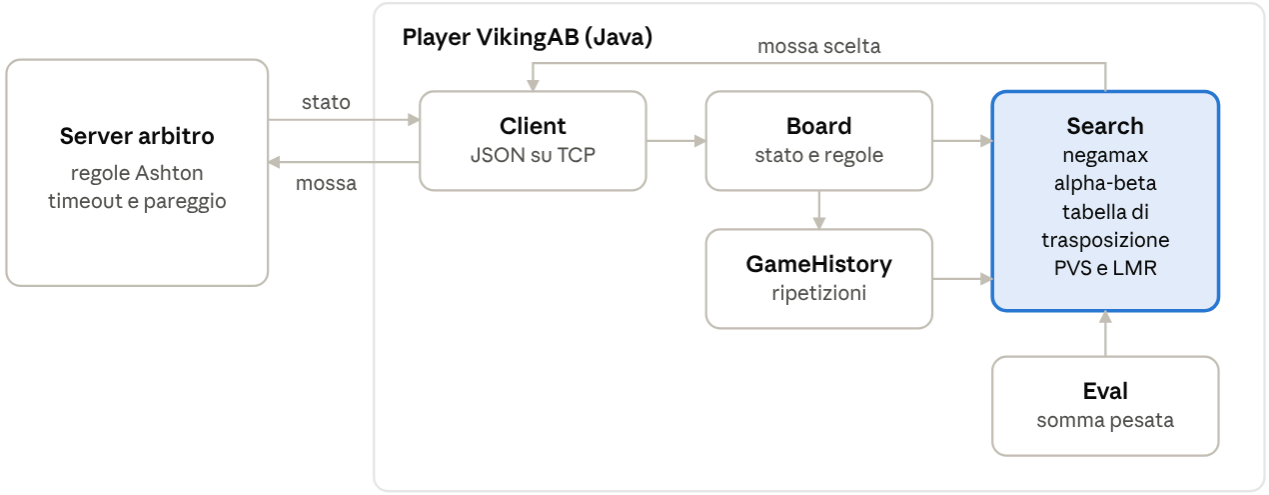
Aspetto	Cosa fa il server
Mossa mancante	Il codice non prevede che chi non ha mosse legali perda.

La regola del pareggio per ripetizione è importante per la strategia: ripetere una posizione senza catture chiude la partita in parità (1 punto), quindi il motore la tratta come una situazione terminale durante la ricerca.

3. Architettura del software

Il player sono sei classi Java: il `Client` parla con il server, `Board` conosce le regole, `Search` sceglie la mossa.

Il client traduce lo stato, la ricerca sceglie la mossa



architettura del player · 6 classi

Il client legge lo stato JSON dal server e lo converte in una `Board`; la `GameHistory` registra le posizioni viste, e `Search` cerca la mossa usando `Eval` per valutare le foglie. La mossa torna al client, che la invia al server.

Classe	Ruolo
`Main`	Legge ruolo, timeout, indirizzo del server e nome; avvia il client
`Client`	Connessione TCP (porta 5800 per il Bianco, 5801 per il Nero), JSON, ciclo di partita, budget di tempo, mossa di emergenza

Classe	Ruolo
`Board`	Stato della partita, generazione delle mosse, catture, vittoria, hash di Zobrist, annullamento delle mosse
`GameHistory`	Posizioni viste dall'ultima cattura, per riprodurre la regola del pareggio del server
`Search`	Negamax con alpha-beta, approfondimento iterativo, tabella di trasposizione, PVS, LMR, ordinamento delle mosse
`Eval`	Valutazione statica come somma pesata di 11 caratteristiche

4. Rappresentazione dello stato e generazione delle mosse

Lo stato è un array di 81 interi, con mosse codificate in un solo intero e aggiornamenti incrementali: serve a cercare milioni di posizioni al secondo (classe `Board`).

- **Scacchiera.** `int[81]` con 0 = vuoto, 1 = bianco, 2 = nero, 3 = re. L'indice è riga×9 + colonna, quindi il trono è l'indice 40.
- **Tabelle precalcolate.** Accampamenti, accampamenti "senza barriera" (i 4 centrali di bordo), caselle di bordo, e `RAYLEN[4][81]`: quanti passi si possono fare da ogni casella verso il bordo in ciascuna delle 4 direzioni. Le direzioni sono gli spostamenti −9, +9, −1, +1.
- **Mossa.** Un intero `da | (a << 7)`: nessun oggetto da allocare durante la ricerca.
- **Generazione.** Da ogni pezzo del lato che muove si scorre lungo i 4 raggi finché si incontra una casella occupata o il trono. Un accampamento ferma la scansione, salvo che il pezzo parta da un accampamento (con il limite di 5 caselle tra accampamenti).
- **Catture.** Si controllano solo le caselle attorno a quella di arrivo, una verifica per direzione, con le stesse condizioni del server (sezione 2).
- **Fare e disfare una mossa.** `make` e `unmake` usano una pila di annullamento (mossa, pezzo, hash, vincitore, posizione del re, pezzi catturati) invece di copiare la scacchiera a ogni nodo.
- **Hash di Zobrist.** Ogni coppia (tipo di pezzo, casella) ha un numero casuale a 64 bit, più uno per il turno. L'hash si aggiorna con uno XOR a ogni mossa ed è la chiave sia della tabella di trasposizione sia del controllo delle ripetizioni. Il seme è fisso, quindi i risultati sono riproducibili.

- **Contatori incrementali.** Numero di soldati bianchi e neri e posizione del re sono aggiornati in `make` / `unmake`, senza ricalcolarli.
- **Raggi aperti del re.** `raysFrom(q, ignore)` conta le direzioni lungo cui dalla casella `q` c'è via libera fino al bordo. Trono, accampamenti e pezzi bloccano; `ignore` è la casella che il re sta lasciando. Una versione precedente sbagliava quando la casella ignorata era il trono: l'abbiamo trovata e corretta prima dei test grandi.

5. L'algoritmo di ricerca

Il cuore del giocatore è un **negamax con potatura alpha-beta**, ripetuto a profondità crescente (approfondimento iterativo) finché non scade il tempo assegnato alla mossa (classe `Search`).

Negamax. È la forma compatta di minimax per giochi a somma zero: ogni posizione ha un valore dal punto di vista di chi muove, e il valore di un nodo è il massimo, tra le mosse, dell'opposto del valore del figlio.

```
v(s) = \max_{m} \bigl( -\, v(s \cdot m) \bigr)
```

Alpha-beta evita di esplorare i rami che non possono cambiare il risultato. Nel caso ideale, con un buon ordinamento delle mosse, permette di raggiungere circa il doppio della profondità di minimax puro a parità di tempo.

Taglio per profondità e valutazione. Quando la profondità residua arriva a zero (o a 95 livelli), il nodo è una foglia e si usa la funzione di valutazione (sezione 6), come previsto dal "cutoff test" di minimax con valutazione euristica visto a lezione. Non c'è ricerca di quiescenza: le foglie si valutano anche se in mezzo a uno scambio di catture.

Posizioni terminali riconosciute nell'albero

- Re bianco con una linea libera fino al bordo e Bianco di turno: vittoria immediata, punteggio `MATE - ply - 1`.
- Mossa che porta alla vittoria (re al bordo o re catturato): `MATE - (ply + 1)`. Sottrarre il livello fa preferire le vittorie più rapide.
- Foglia con il Nero di turno che può catturare il re subito: conta come vittoria del Nero.
- Nessuna mossa legale: sconfitta per chi è di turno.
- Posizione già vista dall'ultima cattura: pareggio, valore 0 (stessa regola del server).

Approfondimento iterativo. Si cerca a profondità 1, poi 2, 3, … (massimo 64). Alla fine di ogni iterazione la mossa migliore passa in testa alla lista, così la successiva parte dalla mossa più promettente. La mossa giocata è quella dell'ultima iterazione completata. Se il

tempo scade a metà di un'iterazione, il risultato parziale si accetta solo se almeno una mossa è stata verificata fino in fondo. La ricerca si ferma prima se trova una vittoria forzata. Il vantaggio è avere sempre una mossa pronta, qualunque sia il tempo disponibile.

Controllo del tempo dentro la ricerca. Ogni 2048 nodi si confronta l'orologio con la scadenza; quando scade, un flag fa uscire dalla ricorsione tutta la ricerca in corso.

Mosse alla radice. Se c'è una sola mossa legale si gioca subito; se una mossa vince all'istante, si gioca quella.

Le ottimizzazioni, in sintesi

Tecnica	Cosa fa	Perché serve
Alpha-beta	Scarta i rami che non possono migliorare il risultato	Riduce enormemente i nodi esplorati
Approfondimento iterativo	Cerca a profondità 1, 2, 3, … fino alla scadenza	Mossa sempre disponibile e ordinamento migliore
Tabella di trasposizione	Memorizza il valore delle posizioni già viste, indicizzate dall'hash	Evita di rifare lavoro quando la stessa posizione si raggiunge con mosse in ordine diverso
PVS (principal variation search)	La prima mossa con finestra piena, le altre con finestra nulla; si ripete la ricerca solo se una mossa supera alpha	Con un buon ordinamento la prima mossa è spesso la migliore, e le finestre nulle sono più veloci
LMR (late move reductions)	Le mosse in coda all'elenco (dalla settima in poi, profondità almeno 3, silenziose) si cercano un livello più in basso; se sembrano buone si ricercano a piena profondità	Risparmia tempo sulle mosse poco promettenti
Ordinamento delle mosse	Prova prima le mosse che hanno più probabilità di causare un taglio	Rende efficace alpha-beta
Controllo ripetizioni	Una posizione già vista dall'ultima cattura vale come pareggio	Riproduce la regola del server

Tabella di trasposizione. Ha 2^22 voci (circa 4,2 milioni), ciascuna con la chiave a 64 bit e un valore compresso in un solo `long` (punteggio, profondità, tipo di limite, generazione, mossa). Occupa circa 64 MiB. I limiti possono essere esatti, inferiori o superiori. I punteggi di vittoria forzata si correggono in base al livello, così restano validi quando la posizione si ritrova a un livello diverso. Una voce più profonda della stessa generazione non viene sovrascritta da una posizione diversa.

Ordinamento delle mosse, in ordine approssimato di priorità

1. La mossa suggerita dalla tabella di trasposizione.
2. Le catture, con priorità maggiore se ne fanno di più.
3. Le mosse del re che creano una o due linee libere verso il bordo.
4. Per il Nero, le mosse che bloccano una linea libera del re.
5. Le due mosse "killer" del livello, cioè quelle che hanno già causato un taglio in posizioni sorelle.
6. Le altre, ordinate con la **history heuristic**: ogni taglio causato da una mossa silenziosa incrementa il suo punteggio di profondità², con un tetto, e a ogni nuova mossa i punteggi si riducono a un ottavo.

La scelta della mossa migliore a ogni passo è un selection sort pigro: se un taglio arriva presto, il resto della lista non viene mai ordinato.

6. La funzione di valutazione

La valutazione è una **somma pesata di 11 caratteristiche** della posizione, esattamente la forma lineare vista a lezione. I pesi attuali sono scelti a mano e non sono ancora stati ottimizzati (classe `Eval`).

```
\mathrm{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)
```

Il valore è calcolato dal punto di vista del Bianco; per il Nero si cambia il segno. La scala parte dal soldato bianco, che vale 100.

Caratteristica	Peso	Effetto	Cosa misura
`wSoldier`	100	a favore del Bianco	Numero di soldati bianchi
`bSoldier`	55	a favore del Nero	Numero di soldati neri
`ray1`	300	a favore del Bianco	Il re ha una sola linea libera fino al bordo

Caratteristica	Peso	Effetto	Cosa misura
`ray2`	1200	a favore del Bianco	Il re ha due o più linee libere: il Nero ne può bloccare una sola per mossa
`threat`	25	a favore del Bianco	Caselle raggiungibili dal re in una mossa da cui parte una linea libera (al massimo 6 contate)
`fork`	150	a favore del Bianco	Caselle raggiungibili dal re in una mossa da cui partono due linee libere (al massimo 3 contate)
`kingMob`	3	a favore del Bianco	Numero di caselle raggiungibili dal re in una mossa
`kingAdj`	30	a favore del Nero	Vicini del re che sono soldati neri o accampamenti
`kingDist`	40	a favore del Bianco	Vicinanza del re al bordo: 40 × (4 − distanza dal bordo più vicino)
`whiteNearKing`	4	a favore del Bianco	Soldati bianchi nella finestra 5×5 attorno al re
`blackNearKing`	6	a favore del Nero	Soldati neri nella finestra 5×5 attorno al re

Come leggerla. Il materiale conta, ma pesa meno del re: per il Bianco ogni linea libera verso il bordo è un passo verso la vittoria, per il Nero ogni pezzo vicino al re è un passo verso la cattura. Il bonus per due linee libere (1200) vale più di dodici soldati, perché è quasi sempre una vittoria forzata. La combinazione "minaccia + doppia minaccia" fa sì che il Bianco cerchi posizioni in cui il re, con una sola mossa, ne crea due.

Perché i pesi sono un vettore. I pesi sono un `int[]`, quindi possono essere sostituiti senza ricompilare. L'idea, coerente con il corso, è ottimizzarli offline con auto-gioco e un algoritmo genetico o PSO. Questa fase non è ancora stata fatta: oggi i valori sono stime ragionate, non misurate.

7. Gestione del tempo e robustezza

Ogni mossa usa circa l'85% del timeout meno mezzo secondo, quindi con timeout 60 la ricerca dura al massimo 50,5 secondi. Lo abbiamo visto nelle prove: la prima mossa del Bianco è arrivata dopo 50.502 ms.

```
t_{\mathrm{mossa}} = \max\bigl(200\ \mathrm{ms},\ 0{,}85 \cdot
\mathrm{timeout} - 500\ \mathrm{ms}\bigr)
```

Il margine (circa 9,5 s su 60) copre l'avvio e la rete e il fatto che il timeout del server ha granularità di 1 secondo.

Altre misure di sicurezza

- **Nessun calcolo nel turno dell'avversario.** Il client legge lo stato e, se non è il suo turno, torna ad aspettare senza fare nulla.
- **Mossa di emergenza.** Se la ricerca solleva un'eccezione, il client gioca la prima mossa legale invece di perdere per timeout.
- **Connessione con ripetizioni.** Se il server non è ancora in ascolto, il client riprova ogni 500 ms per circa 2 minuti.
- **Solo mosse legali.** Le mosse vengono dal generatore, che è stato confrontato con il server (sezione 8), non da una nostra interpretazione delle regole.
- **Memoria.** Il processo parte con un massimo di 2 GB di heap (`-Xmx2g`); la tabella di trasposizione ne usa circa 64 MiB.

Protocollo di rete. Il client apre una connessione TCP sulla porta 5800 (Bianco) o 5801 (Nero) e scambia messaggi composti da 4 byte di lunghezza (big-endian) seguiti da una stringa JSON in UTF-8. Invia prima il proprio nome, poi ripete: legge lo stato (`board` e `turn`); se il turno è suo calcola e risponde con `{from, to, turn}`. Quando `turn` diventa `WHITEWIN`, `BLACKWIN` o `DRAW`, la partita finisce e il client stampa il risultato.

Cosa si vede a console. Per ogni mossa una riga con numero, mossa, profondità raggiunta (`d`), punteggio (`s`), nodi (`n`) e tempo (`t`). Un esempio dalle prove: `#1 e6f6 d=8 s=-61 n=30586880 t=50502ms`.

8. Verifica e test svolti

La correttezza delle regole è stata verificata contro il motore del server con circa 20 milioni di mosse confrontate e 0 differenze; la forza di gioco, invece, non è stata ancora misurata con un metodo solido.

Test	Cosa verifica	Esito
Confronto con il motore del server (`DiffTest`)	Per ogni coppia (da, a) possibile del lato di turno: legalità e stato risultante, su partite casuali e posizioni casuali. 3 serie di 600 partite + 12.000 posizioni	Circa 20 milioni di coppie, 6,8 milioni di mosse legali, circa 300.000 catture, circa 18.000 catture del re, circa 32.000 vittorie del Bianco: 0 differenze
Test di mutazione del confronto	Si alterano di proposito 4 regole del nostro motore per vedere se il confronto se ne accorge	Differenze rilevate in tutti e 4 i casi: il test non è "cieco"
Auto-gioco con il server vero come arbitro (`SelfPlay`)	Partite del player contro se stesso, con 4 mosse iniziali casuali	0 mosse illegali; circa 1,85 milioni di nodi/s; profondità media circa 7,8 a 300 ms per mossa
Partita completa via socket con server e giocatore casuale del repo	Protocollo di rete, ruoli, fine partita	Vittorie sia da Bianco (con timeout 10 s, mossa a 8.001 ms) sia da Nero; la partita giocatore contro giocatore è arrivata alla fine
Argomento del ruolo	`WHITE`, `white`, `wHitE`	Tutti accettati

Un errore trovato dai test. Il calcolo dei raggi aperti del re sbagliava quando la casella ignorata era il trono. È stato corretto prima dei test grandi.

Cosa i test non dicono

- Battere un giocatore casuale non è una misura di forza.
- Le misure di velocità e profondità sono state fatte nel nostro ambiente di sviluppo. Nella VM ufficiale con 4 CPU i numeri possono essere diversi; sul computer Linux di un membro del gruppo la prima mossa a timeout 60 è arrivata a profondità 8 con circa 30,6 milioni di nodi in 50,5 s.
- In alcune prove di auto-gioco con i pesi predefiniti il Nero ha vinto più spesso del Bianco (ad esempio 4 partite su 6 in una prova). Il campione è troppo piccolo per concludere qualcosa, ma indica che i pesi vanno calibrati per colore.
- La VM ufficiale non è ancora stata provata, e la sua versione di Java non è nota. Il jar è compilato per Java 8; tutti i nostri test sono stati fatti con Java 21 e non l'abbiamo provato su un vero Java 8.

9. Il collegamento con il corso

Il nucleo del player è fatto con algoritmi delle slide (minimax con cutoff e valutazione, alpha-beta, approfondimento iterativo); sopra a questo abbiamo aggiunto tecniche standard dei programmi per giochi da tavolo che il corso non richiedeva.

Dal corso	Dove nel codice	Come lo usiamo
MIN-MAX con CUTOFF-TEST ed EVAL (slide `9-Games`)	`Search.negamax`, `Eval`	Stessa idea in forma negamax. Il cutoff test è la profondità residua; EVAL è la valutazione lineare
Potatura alpha-beta (`9-Games`)	`Search.negamax`	Si tagliano i rami quando alpha ⩾ beta
Funzione di valutazione come somma pesata di caratteristiche (`9-Games`)	`Eval.evalWhite`	11 caratteristiche con pesi
Approfondimento iterativo (`4-StriRicNonInf`)	`Search.think`	Profondità 1, 2, 3, … fino alla scadenza del tempo, ordinando le mosse con i risultati precedenti
PSO (`8-SwarmIntelligence`)	Non ancora implementato	Candidato per ottimizzare offline i pesi con l'auto-gioco come funzione di fitness

Oltre al corso. Tabella di trasposizione con hash di Zobrist, principal variation search, late move reductions, mosse killer e history heuristic. Sono tecniche note per rendere alpha-beta più veloce, ma conviene saperle spiegare bene, perché non sono il nucleo di quanto visto a lezione.

Perché non altri algoritmi del corso

- **A* e ricerca informata.** Cercano un cammino di costo minimo verso un obiettivo per un solo agente. Qui l'avversario sceglie a sua volta le mosse, quindi serve una ricerca avversaria.
- **Pianificazione (STRIPS, POP, Graphplan).** Costruisce un piano in un mondo che cambia solo per le azioni dell'agente. Un piano non regge a un avversario che risponde a ogni mossa.
- **Algoritmi di ricerca locale.** Non danno una mossa da giocare nella partita; hanno senso, al massimo, per ottimizzare i pesi fuori dalla partita.

10. Limiti noti e miglioramenti possibili

I limiti principali sono tre: pesi non tarati, nessuna ricerca di quiescenza e un solo thread, e la presentazione deve dirli apertamente.

Limite	Conseguenza	Possibile miglioramento
Pesi scelti a mano	La forza reale è sconosciuta; in alcune prove il Nero vince più del Bianco	Ottimizzazione offline (PSO o algoritmo evolutivo) con auto-gioco, con pesi separati per colore e colori bilanciati
Nessuna ricerca di quiescenza	Le foglie vengono valutate anche nel mezzo di uno scambio di catture	Estendere la ricerca sulle sole catture e sulle minacce al re
Un solo thread	Delle 4 CPU previste se ne usa una	Ricerca parallela (per esempio sulle mosse alla radice)
Nessun libro di aperture	Le prime mosse costano quanto le altre (circa 50 s nella prova)	Mosse precalcolate per le prime posizioni
Tempo usato quasi per intero a ogni mossa	Margine di circa 9,5 s su 60	Fermarsi prima quando la mossa migliore è stabile
Regole del torneo che possono cambiare	Il motore replica il server di oggi	Controllare il sito del corso e le issue su GitHub prima della consegna
Verifica solo nel nostro ambiente	Velocità e versione di Java della VM non confermate	Provare nella VM ufficiale (sezione 12)

Vincoli della consegna rispettati. Nessun accesso alla rete a runtime, nessuna GPU e nessun calcolo durante il turno dell'avversario.

11. Domande probabili alla presentazione

Sono domande che un docente può fare su questo progetto, con la risposta che i dati reali permettono di dare. Chi presenta deve saper rispondere a tutte, non solo alla propria parte.

Perché avete scelto minimax con alpha-beta? Tablut è un gioco a due giocatori, deterministico, a informazione perfetta e a somma zero, che è il caso per cui minimax con valutazione euristica e alpha-beta è stato pensato. La pianificazione e A* non gestiscono un avversario che risponde alle mosse.

Che differenza c'è tra minimax e negamax? Nessuna nel risultato: il negamax sfrutta il fatto che il valore per un giocatore è l'opposto del valore per l'altro, e scrive una sola funzione invece di due (MAX e MIN).

Come funziona alpha-beta e quanto fa risparmiare? Si tengono due limiti: alpha, il miglior valore già garantito a chi muove, e beta, quello garantito all'avversario. Quando alpha ≥ beta il resto dei figli non può cambiare il risultato e si taglia. Il risparmio dipende dall'ordinamento delle mosse; non abbiamo misurato il fattore esatto sul nostro programma, che dipende dalla posizione.

Come avete scelto la funzione di valutazione e i pesi? Le caratteristiche vengono dalle regole del gioco: materiale, linee libere del re verso il bordo, mobilità del re, pezzi nemici attorno al re. I pesi sono stati scelti a mano e **non sono stati ottimizzati**. Dirlo apertamente è meglio che far credere il contrario.

Come gestite il tempo? Con l'approfondimento iterativo: si cerca a profondità crescente e si usa il risultato dell'ultima iterazione completata. Il budget è l'85% del timeout meno 0,5 s, e ogni 2048 nodi si controlla l'orologio.

A che profondità arrivate? Dipende dalla posizione. Alla prima mossa del Bianco con 60 s abbiamo visto profondità 8 e circa 30,6 milioni di nodi. In auto-gioco a 300 ms per mossa la media era circa 7,8. I due numeri vengono da condizioni diverse e non vanno confrontati direttamente: non abbiamo misurato la profondità in funzione del tempo.

A cosa serve la tabella di trasposizione? A riutilizzare il valore di una posizione già incontrata, anche se raggiunta con mosse in ordine diverso. La posizione è identificata dall'hash di Zobrist, aggiornato con uno XOR a ogni mossa. Fornisce inoltre la mossa da provare per prima.

Come sapete che le mosse sono sempre legali? Le regole sono state portate dal codice del server e confrontate con esso su circa 20 milioni di coppie (da, a), con 0 differenze; un test di mutazione ha mostrato che il confronto rileva davvero gli errori. Inoltre, se la ricerca fallisce con un'eccezione, il client gioca la prima mossa legale invece di perdere per timeout.

Come gestite il pareggio per ripetizione? Il server dichiara pareggio quando uno stato si ripete dall'ultima cattura. Il nostro client tiene l'elenco degli stati visti e la ricerca li tratta come pareggio con valore 0, così, quando è in vantaggio, evita di ripetere posizioni; quando è in svantaggio può cercare il pareggio.

Quali sono i punti deboli? Pesi non ottimizzati, nessuna ricerca di quiescenza, un solo thread e nessun libro di aperture (sezione 10).

Perché non una rete neurale o apprendimento per rinforzo? La consegna ammette solo 4 CPU, nessuna GPU e nessuna rete a runtime, e il corso tratta ricerca e ottimizzazione. Non abbiamo provato queste strade.

Cosa fareste con più tempo? Calibrare i pesi con auto-gioco (PSO o algoritmo evolutivo), aggiungere la ricerca di quiescenza e usare tutte e 4 le CPU.

12. Come eseguire, ricompilare e consegnare

Il comando di avvio richiesto dalla consegna è `./runmyplayer.sh RUOLO TIMEOUT IP_SERVER`, per esempio `./runmyplayer.sh WHITE 60 192.168.20.101`; il ruolo è la parola intera e non distingue maiuscole e minuscole.

Contenuto della cartella `tablut`

File	A cosa serve
`runmyplayer.sh`	Script di avvio richiesto dalla consegna: lancia `java -Xmx2g -jar TablutPlayer.jar` con i 3 parametri
`TablutPlayer.jar`	Il player compilato (Java 8 o successivo), con la libreria Gson inclusa
`gson-2.2.2.jar`	Libreria JSON, necessaria solo per ricompilare
`build.sh`	Ricompila e rigenera il jar (serve un JDK; usa `javac --release 8`)
`src/tablutai/`	Sorgenti: `Board`, `Eval`, `Search`, `GameHistory`, `Client`, `Main`
`test/`	`DiffTest` (confronto con il motore del server) e `SelfPlay` (partite con l'arbitro reale); richiedono le classi del server

Installazione nella VM ufficiale

1. Copiare il contenuto di `tablut` in `/home/tablut/tablut/`, togliendo il vecchio player casuale (tenerne una copia per i test).
2. Dare il permesso di esecuzione: `chmod +x /home/tablut/tablut/runmyplayer.sh`. Se compare un errore di interprete o `^M`, lo script ha i finali di riga di Windows: `sed -i 's/\r$//' runmyplayer.sh`.
3. Non modificare utente, password e impostazioni SSH della VM.

Prova rapida con server e avversario

1. Dal repo del server (cartella `Tablut`), dopo aver compilato con `ant`, avviare il server con timeout breve. `ant server` non accetta opzioni, quindi si usa direttamente Java: `java -cp "lib/gson-2.2.2.jar:lib/commons-cli-1.4.jar:build" it.unibo.ai.didattica.competition.tablut.server.Server -t 10`.

2. In un altro terminale: `./runmyplayer.sh WHITE 10 localhost`.
3. In un terzo terminale l'avversario, per esempio `ant randomblack`, oppure una seconda copia del nostro player con `BLACK 10 localhost`.
4. Il timeout del player deve essere uguale a quello del server. Con timeout 10 ogni mossa dura circa 8 secondi; con 60 secondi si arriva a circa 50 secondi per mossa, quindi due copie del player che giocano tra loro impiegano più di un'ora.
5. A fine partita il player stampa `Game over: ... -> WIN/LOSS/DRAW`.

Senza la VM. Su un computer con Java installato si può lanciare direttamente `java -Xmx2g -jar TablutPlayer.jar WHITE 60 localhost`; senza un server in ascolto il player riprova a collegarsi per circa 2 minuti e poi termina.

Cambiare il nome del giocatore. Si modifica `DEFAULT_NAME` in `src/tablutai/Main.java` e si lancia `./build.sh`. Il server ne conserva solo i primi 10 caratteri alfanumerici.

Consegna (secondo la presentazione del corso)

- Entro il 22 novembre, ore 23:59 italiane, su Virtuale: un file `.txt` con il link OneDrive all'immagine della VM, il comando per avviare il player ed eventuali parametri opzionali.
- Poi: due questionari anonimi e la presentazione finale, a cui devono partecipare tutti i membri del gruppo.
- Le regole possono cambiare: controllare il sito del corso e le issue del repository su GitHub.
- Verificare la regola del corso sull'uso di strumenti di intelligenza artificiale per scrivere il codice e dichiararlo se richiesto.